تقرير التقدّم: تحسين نظام كشف التناقضات

مسارا Code vs Doc و Code vs Code — الخطوات 0 و 1 و 2 من الخطة التنفيذية

التاريخ: 25-10-2026 المشروع: Senior-project1ATDG / Ai_project آخر commit: `722d170`

3	21/21	2	581
خطوات منجزة (0 و 1 و 2)	اختبار ناجح بالـ Docker	أخطاء إنتاج انصلحت	دالة حقيقية اتقارنت قديم/جديد

المحتويات

1. الخطط الثلاث والتقييم المبدئي
2. الخطة التنفيذية (14 خطوة)
3. الخطوة 0: التجهيز والتحقق من الـ baseline
4. الخطوة 1: استخراج جسم الدالة ومعلوماتها
5. الخطوة 2: تحديث الـ schemas وإصلاح category
6. بيئة الاختبار (Docker)
7. ملخص الاختبارات
8. المشاكل المكتشفة وحالتها
9. سجل الـ commits والملفات
10. الخطوات الجاية

1. الخطط الثلاث والتقييم المبدئي

انطلق العمل من ثلاث خطط مكتوبة، كل وحدة مبنية على أوراق علمية:

الخطة	الأوراق	الفكرة الأساسية
تحسين مسار Code vs Doc	DocPrism (ISSTA 2026)، METAMON (arXiv 2025)	تقسيم التوثيق وربط كل جزء بعنصره، تحليل دلالي بالـ LLM، تصنيف LCEF الرباعي، تجاهل الـ Under-Promise، درجات ثقة وعتبات قرار، و Fallback عند فشل الـ LLM.
تحسين مسار Code vs Code	AexPy (ISSRE 2022)، Safer Builders (MSR 2026)، NPM Breaking Changes (TOSEM 2025)	مقارنة التوقيع والجسم دائماً، مصفوفة قرار (Signature × Body)، استدعاء الـ LLM بس للتغييرات السلوكية، تصنيف خطورة بمستويات AexPy الأربعة.
خطة إثبات الدقة	DocPrism، Cascade (FSE 2026)، C4RLLaMA	قياس Precision و Recall و Specificity و F1 و PFP و Flag Rate على عينات جاهزة، ومقارنة النظام القديم مع الجديد.

أهم الملاحظات على الخطط قبل التنفيذ

1. أساس مشترك ناقص: جسم الدالة

خطة Code vs Code قائمة على "افحص الـ Body دائماً"، وخطة الإثبات معتمدة على Cascade يلي تناقضاتها سلوكية جوا الجسم. بس `MethodDetail` ما كان فيه أي حقل للجسم، والـ processors ما كانت تستخرجه. لهيك صار هاد **الخطوة 1**.

- **Code vs Doc ما فيه LLM أصلاً:** كله regex بـ `code_doc_consistency.py` . والخطة كانت بتقترح تعديل `semantic_analyzer.py` ، وهاد ملف Code vs Code. القرار: محلّل دلالي منفصل للتوثيق.
- **ما منلغي المطابقة القديمة كلها:** فحوصات التوثيق المنظّم (متل raises مقابل return None) دقيقة وبدون تكلفة، فبتضل طبقة أولى و Fallback.
- **الدوال غير الموثّقة:** حالياً كل وحدة بتنحسب تناقض، وهاد مصدر رئيسي للإنذارات الكاذبة. القرار: بتنتقل لإحصائيات التغطية.
- **بناء "قبل وبعد لكل hunk":** فكرة Safer Builders معمولة للـ commits، ونظامنا بياخد نسختين كاملتين. القرار: مؤجّلة.
- **خطة الإثبات:** بتحتاج محوّل (adapter) من صيغة عينات DocPrism و Cascade لصيغة مدخلات نظامنا، وهاد مش مذكور بالخطة.

2. الخطة التنفيذية (14 خطوة)

الخطوة	المسار	العمل	الحالة
0	تجهيز	git، والتحقق من الـ baseline المجمّد	✅ منجزة
1	أساس مشترك	استخراج الـ body والمعاملات والتوثيق، و async، والـ constructors	✅ منجزة
2	أساس مشترك	تحديث الـ schemas، وإصلاح `category`	✅ منجزة
3	Code vs Doc	تقسيم التوثيق وربط كل جزء بعنصره (DocSplitter)	التالية
4	Code vs Doc	`CODE_DOC_PROMPT`	—
5	Code vs Doc	`DocSemanticAnalyzer` : Fallback، فلترة خارجية، عتبة ثقة	—
6	Code vs Doc	الربط بـ `compare()` ، والدوال غير الموثّقة لإحصائيات التغطية	—
7	Code vs Code	`SignatureComparer` و `ElementTypeClassifier` ، وإصلاح RenameDetector	—
8	Code vs Code	مقارنة الـ Body كخطوة إلزامية	—
9	Code vs Code	`change_classifier.py` ومصفوفة القرار	—
10	Code vs Code	`severity_classifier.py` (مستويات AexPy)	—
11	Code vs Code	منع استدعاء الـ LLM للتغييرات الهيكلية، وعدّاد للاستدعاءات الموفّرة	—
12	إثبات	Adapters لعينات DocPrism و Cascade	—
13	إثبات	الـ Ground Truth والوسم اليدوي	—
14	إثبات	تشغيل القديم والجديد، المقاييس، التقرير	—

قاعدة ثابتة بكل الخطوات

كل التعديلات بتنعمل على كود الإنتاج (`core_ai/`). النسخة المجمّدة `evaluation/legacy_baseline/` ممنوع تنلمس، لأنها النظام القديم يلي منقارن فيه بالتقرير. بنتحقق منها بأول وآخر كل خطوة بأمر `python -m evaluation.harness.snapshot`.

3. الخطوة 0: التجهيز والتحقق من الـ baseline

3.1 شو كان معمول من قبل (المرحلة 0)

المكوّن	الوظيفة
`legacy_baseline/`	نسخة مجمّدة من `conflict_detection` و `language_processors`. انجمّدت الـ processors كمان، لأنه نتيجة الكشف بتعتمد على شو بيطلّعوا.
`snapshot.py` و `MANIFEST.json`	بصمة sha256 لكل ملف وبصمة إجمالية، لإثبات إنه النسخة ما انلمست، مع سجل للتعديلات المبرّرة (amendments).
`llm_recorder.py`	LLM حتمي: temperature 0، seed 42، بدون cache، موديل مثبّت، وتسجيل كل استدعاء.
`run_baseline.py`	بيشغّل النسخة المجمّدة على العينات وبيحفظ النتائج read-only مع البصمات.
`labeling/`	أدلة الوسم اليدوي، وقوالب CSV، وحساب Kappa.

التعديلان الموثّقان على النسخة المجمّدة: استعمال الـ LLM الحتمي، وإصلاح `category` لحتى ما يوقع الـ baseline على كل عينة.

3.2 شو عملنا بالخطوة 0

الفحص	النتيجة
بصمة النسخة المجمّدة	✅ **ok: a8649b22… — ما انلمست**
تشغيل Code vs Doc على عينة تجريبية	اشتغل بدون LLM. طلّع إنه `add` مش موثّقة، بس **فوّت تناقض حقيقي**: التوثيق بيقول "Raises KeyError" والكود بيرجّع Optional.
تشغيل Code vs Code بدون إنترنت	الجزء الهيكلي اشتغل بدون crash، و `semantic_failed=True` متل المتوقع.
إعداد git	مستودع محلي مع `.gitignore` بيستثني `.env` و `node_modules` و `__pycache__` و `db.sqlite3` و `staticfiles` والملف المضغوط. أول commit: `c35580a` (780 ملف).

ملاحظتين من الخطوة 0

- بـ `settings.py` في مفتاح OpenRouter مكتوب كقيمة افتراضية. القرار: يضل متل ما هو لأنه المستودع محلي. **لازم ما ينرفع المستودع على GitHub عام بهالحالة.**
- ممكن يكون كان في `.gitignore` قديم، وانكتب فوقه وقت إعداد git.

4. الخطوة 1: استخراج جسم الدالة ومعلوماتها

commit: `84e5ac5` — تقرير الاختبارات: `core_ai/tests/reports/step1_method_details.md`

4.1 المشكلة

- كل دالة كان ينحفظلها: الاسم، والتوقيع كنص، والـ visibility، و abstract/constructor. **بدون جسم.**
- معلومات المعاملات ناقصة: بايثون ما بيسجّل القيم الافتراضية ولا `*args` و `**kwargs` . جافا بتطلّع المصفوفة `Object` وبتضيّع الـ varargs.
- دوال ضايعة من المخطط: دوال `async` بالبايثون، والـ constructors بالجافا.

4.2 الحقول الجديدة لكل دالة

الحقل	المحتوى
`body`	جسم الدالة بدون تعليقات وبدون docstring (حد أعلى 20 ألف حرف)
`body_hash`	بصمة للجسم بعد التطبيع: التعليقات والمسافات ما بتغيّرها، وتغيير المنطق بيغيّرها. بتنحسب على الجسم الكامل.
`parameters`	لكل معامل: name و type و default و kind (positional_only, positional_or_keyword, keyword_only, var_positional, var_keyword)
`return_type`	نوع الإرجاع المكتوب
`decorators`	متل `@staticmethod` و `@property` و `@Override`
`docstring`	docstring بالبايثون، أو Javadoc/JSDoc (`/** */`)، أو C# (`///`)
`is_async` و `line_start` و `line_end`	معلومات إضافية

> **قرار تصميم**
> حقل `signature` القديم ضل **حرف بحرف** متل ما هو، لأنه الواجهة الأمامية و `FingerprintBuilder` بيعتمدوا عليه. المعلومات الصحيحة صارت بالحقول الجديدة.

4.3 الملفات المعدّلة

الملف	التعديل
`method_details.py` (جديد)	أدوات مشتركة: تطبيع الجسم وحساب البصمة، والحد الأعلى للطول، وقراءة التوثيق يلي فوق الدالة.
`python_processor.py`	دالة `_method_details` بتستعمل `ast` لاستخراج كل الحقول. ودعم `AsyncFunctionDef` .

`java_processor.py`	دالة `_method_details` بتستعمل tree-sitter. ودالة جديدة `_extract_constructor_info` لإضافة الـ constructors.
`tree_sitter_processor.py`	نفس الحقول لباقي اللغات (JS و TS و C# ...). مع قراءة معاملات JS، و `async` بالـ C#.
`schemas.py`	`ParameterDetail` ، والحقول الجديدة على `MethodDetail` ، **كلها اختيارية** للتوافق مع التحليلات المحفوظة.
`ast_unifier.py`	تمرير الحقول الجديدة.
`semantic_analyzer.py`	البيانات المرسلة للـ LLM محصورة بالحقول الستة القديمة، فالـ prompt ما تغيّر.
`structural_diff.py`	إصلاح: `parameters=None` (بالتحليلات القديمة) كان رح يوقّع المقارنة.

4.4 مثال: البصمة

نفس البصمة: التعليق والمسافات والـ docstring مختلفين، والمنطق نفسه.

النسخة A	النسخة B

`def get(self, uid):`
`  """Return the user."""`
`  # lookup`
`  return self.store.get(uid)`
	`def get(self, uid):`
	`  """Fetch the user."""`
	`  return  self.store.get( uid )  # trailing`

بصمة مختلفة: السطر صار `return self.store[uid]` ، يعني الدالة صارت بترمي `KeyError` بدل ما ترجّع `None` .

4.5 التحقق

الفحص	النتيجة
12 اختبار وحدة	✅ 12/12 بالـ Docker
بايثون: قديم مقابل جديد	147 ملف حقيقي و 581 دالة: **كل الحقول القديمة متطابقة 100%**، والكلاسات والعلاقات متطابقة
جافا: قديم مقابل جديد	كل الحقول القديمة متطابقة، والفرق الوحيد 4 constructors انضافوا
JS و TS و C#	متطابق. الفحص كشف 3 نواقص وانصلحت: معاملات JS، و async بالـ C#، و توثيق `///`
المسار الكامل بدون إنترنت	Code vs Code و Code vs Doc اشتغلوا

> **ملاحظة**
>
> غيّرنا `force=False` لـ `force` وغيّرنا سطر منطقي، والكود مقابل الكود طلّع **صفر تغييرات**. الحقول موجودة بس لسا ما حدا عم يقارنها، وهاد شغل الخطوتين 7 و 8.

5. الخطوة 2: تحديث الـ schemas وإصلاح category

commit: `722d170` — تقرير الاختبارات: `core_ai/tests/reports/step2_schemas.md`

5.1 الخطأ الخطير بكود الإنتاج

أي تغيير هيكلي كان يوقّع مسار Code vs Code كله

حقل `category` كان معرّف إنه بيقبل 3 قيم بس: `syntactic` و `semantic` و `behavioral` . بينما `structural_diff.py` بيبعت 13 قيمة متل `method_removed` و `class_added` . النتيجة: `ValidationError` مع حذف أو إضافة أي دالة، أو تغيير أي توقيع.

الحالة	قبل الخطوة 2	بعد الخطوة 2
حذف دالة	ValidationError	method_removed
إعادة تسمية	ValidationError	renamed

الإصلاح: `category` صار نص حر بيحمل اسم نمط التغيير، وهاد بالزبط يلي رح نحتاجه بالخطوة 10 لنربط كل نمط بمستوى AexPy.

5.2 الإضافات

الإضافة	القيم	تُستعمل بـ
`ChangeType.RENAMED`	`renamed`	هلق — بدل ما تنحسب إعادة التسمية behavior_change
`DocConsistencyLabel`	Consistent، Direct_Mismatch، Over_Promise، Under_Promise	الخطوات 3–6 (DocPrism)
`ChangeDecision`	structural_only، structural_and_behavioral، behavioral_candidate، no_change	الخطوة 9
`RiskLevel`	critical، warning، informational	الخطوة 10 (AexPy)
حقول على `ConflictItem`	`doc_label` ، `is_flagged` ، `evidence` ، `decision` ، `risk_level` — كلها اختيارية	الخطوات الجاية

قرار تصميم

حقل `severity` وقيمه (critical, high, medium, low, info) **ما تغيّروا**، لأنه الواجهة الأمامية بتلوّن فيهم صفحات Code vs Code و Code vs Doc. مستوى AexPy صار بحقل جديد `risk_level` جنبه.

5.3 التحقق

الاختبار	قبل	بعد

حذف دالة من كلاس	ValidationError	method_removed
8 أنماط تغيير هيكلي مع بعض	crash	الثمانية طلعوا
المسار الكامل عبر ConflictOrchestrator	crash	success
إعادة تسمية fetch_user → get_user	crash	renamed
تقرير قديم بدون الحقول الجديدة	✅	✅ بينقرأ
الحقول الجديدة بتنحفظ كنص وبترجع تنقرأ	—	✅
قيم severity ما تغيّرت	✅	✅
أسماء DocPrism مطابقة لدليل الوسم	—	✅

6. بيئة الاختبار (Docker)

على جهاز التطوير، tree-sitter نسخة 0.25، بينما المشروع مثبّت على 0.20. بالنسخة الجديدة ما في `set_language`، فـ **معالج الجافا ما بيشتغل على الجهاز أصلاً، حتى النسخة القديمة منه**. لهيك كل الاختبارات النهائية بتنعمل بحاوية `ai_web`، يلي فيها البيئة الحقيقية:

المكوّن	جهاز التطوير	حاوية ai_web
Python	3.13	3.10.20
tree-sitter	0.25.2	0.20.4
tree-sitter-languages	غير مثبّت	1.6.1
معالج الجافا	ما بيشتغل	بيشتغل

أمر التشغيل (حاوية مؤقتة، بدون قواعد بيانات، بتنمسح لحالها، والصورة ما بتتغيّر):

```
MSYS_NO_PATHCONV=1 docker compose run --rm --no-deps -T --entrypoint sh ai_web -c \
  "pip install -q pytest; python -m pytest core_ai/tests -v; python -m evaluation.harness.snapshot"
```

- `--entrypoint` ضروري، لأنه سكربت التشغيل بينتظر MongoDB وبيفشل.
- مجلد `Ai_project` مربوط بالحاوية كـ volume، فالتعديلات بتنشاف بدون إعادة بناء.

7. ملخص الاختبارات

الملف	العدد	بيغطّي
`core_ai/tests/test_method_details.py`	12	بايثون (6)، جافا (3)، توافق النظام (3)
`core_ai/tests/test_schemas_step2.py`	9	إصلاح (1) RENAMED، (3) category، الحقول والتوافق (5)
المجموع	**21**	21/21 نجحت بالـ Docker، والنسخة المجمّدة ما انلمست

8. المشاكل المكتشفة وحالتها

#	المشكلة	بالـ baseline؟	الحالة
1	أي تغيير هيكلي بيوقّع Code vs Code (`category`)	انصلح بالـ amendment	✅ انصلح (الخطوة 2)
2	دوال async بالبايثون والـ constructors بالجافا ضايعين من المخطط	نعم	✅ انصلح (الخطوة 1)
3	معاملات ناقصة: القيم الافتراضية، و *args، والمصفوفات، و varargs	نعم	✅ انصلح بالحقول الجديدة (الخطوة 1)

4	Code vs Doc بيفوّت تناقض "raises مقابل return None" بالتوثيق النصي	نعم	الخطوات 3–6
5	إعادة التسمية بتنعرض مرتين (refactoring + renamed)	نعم	الخطوة 7
6	إعادة تسمية وهمية: إضافة `b()` مع بقاء `a()` بتطلع "a انعاد تسميتها لـ b"	نعم	الخطوة 7
7	تغيير القيمة الافتراضية أو الجسم ما بينكشف	نعم	الخطوتين 7 و 8
8	الـ prompt بيطلب `type=RENAMED` بأحرف كبيرة، والـ parser بيتوقّع `renamed`	نعم	الخطوة 11
9	بيئة جهاز التطوير ما بتشغّل معالج الجافا (نسخة tree-sitter)	نعم	الحل: Docker للتقييم

أهمية هالجدول للتقييم

المشاكل 4–8 موجودة بالنسخة المجمّدة، فهي نقاط محددة رح نقيس فيها التحسّن بجدول "القديم مقابل الجديد".

9. سجل الـ commits والملفات

commit	الوصف
`c35580a`	Baseline before Phase 1 (frozen legacy + evaluation harness)
`84e5ac5`	Step 1: extract method body, parameters, docstring, async methods and Java constructors
`722d170`	Step 2: fix category crash, add RENAMED and schema fields for DocPrism/AexPy

ملفات جديدة

- `core_ai/language_processors/method_details.py`
- `core_ai/tests/test_method_details.py` و `core_ai/tests/test_schemas_step2.py`
- `core_ai/tests/reports/step1_method_details.md` و `step2_schemas.md`

ملفات معدّلة

- `core_ai/language_processors/` : `python_processor.py` ، `java_processor.py` ، `tree_sitter_processor.py`
- `core_ai/services/conflict_detection/` : `schemas.py` ، `ast_unifier.py` ، `semantic_analyzer.py` ، `structural_diff.py`

ما انلمس: `evaluation/legacy_baseline/` ، والواجهة الأمامية، و `settings.py` .

10. الخطوات الجاية

الخطوة 3: تقسيم التوثيق (DocSplitter)

بدل ما ينبعت التوثيق كله دفعة وحدة، منقسّمه لأجزاء ومنربط كل جزء بالكلاس أو الدالة يلي بيحكي عنها. هاد أساس فكرة DocPrism إنه كل دالة بتنقارن بالتوثيق تبعها بس. قبل التطبيق، منفحص شكل التوثيق يلي بيولّده النظام فعلياً، لأنه التقسيم معتمد عليه.

بعدها

- **الخطوات 4–6:** prompt مخصص، ومحلّل دلالي للتوثيق مع Fallback وعتبة ثقة، والربط مع نقل الدوال غير الموثّقة لإحصائيات التغطية.
- **الخطوات 7–11:** مقارنة التوقيع والجسم، ومصفوفة القرار، وتصنيف الخطورة، وتقليل استدعاءات الـ LLM.
- **الخطوات 12–14:** التقييم على DocPrism و Cascade، وجدول القديم مقابل الجديد.